기숙사 세탁기 혼잡 예측 웹앱

AI 빨래 도우미

대전과학고등학교 2학년 · 정보과학프로젝트

2407 박민석

사이트	https://one-iota-62.vercel.app/
GitHub	https://github.com/pms09/-

1. 프로젝트 개요

문제 상황. 기숙사 세탁실에는 세탁기가 6대뿐인데, 학생들은 세탁기가 비어 있는지 모른 채 빨래 바구니를 들고 내려간다. 자리가 없으면 그대로 다시 올라와야 하고, 저녁 시간대에는 이런 헛걸음이 반복된다.

해결 아이디어. 센서나 IoT 장비를 설치하지 않고도, **학생들이 직접 남긴 기록을 모으면** 요일·시간대별 혼잡 패턴을 알 수 있다. 이 통계를 **Google Gemini**에 함께 보내서, 사용자가 자연어로 물어보면 "언제 가면 좋을지"를 이유와 함께 문장으로 답해 주는 웹앱을 만들었다.

핵심 설계 결정

항목	선택	이유
데이터 수집	학생 수동 입력	센서·IoT 없이 구현 가능. 기숙사 현실에 맞음
예측 방식	요일·시간대별 통계 평균	실시간 대기열이 아니라 '보통 이 시간엔 이렇다'를 알려줌
AI 역할	통계 해석 + 자연어 추천	숫자를 사람이 읽는 문장으로 바꿔 줌
저장소	Supabase (Postgres)	배포 환경에서 기록이 영구 보존됨

왜 '실시간'이 아니라 '예측'인가? 센서가 없으므로 지금 이 순간 몇 대가 비었는지는 알 수 없다. 대신 과거에 같은 요일·같은 시간대에 남겨진 기록들의 평균을 내어 **여유 / 보통 / 혼잡** 3단계로 보여 준다. 기록이 쌓일수록 예측이 정확해지는 구조다.

2. 기능 구성 — 메인 1개 · 서브 4개

메인 기능

기능	설명
AI 빨래 도우미	사용자의 자연어 질문과 요일·시간대별 혼잡도 통계를 함께 Google Gemini API에 보내, 추천 시간과 그 **근거**를 한국어 문장으로 받아 보여 준다.

서브 기능

번호	기능	설명
서브 1	시간대별 혼잡도 그래프	선택한 요일의 06~23시 혼잡도를 여유(초록)·보통(노랑)·혼잡(빨강) 막대그래프로 표시
서브 2	현황 기록 + DB 저장	'지금 비어 있는 세탁기 수'를 입력하면 Supabase에 영구 저장되고, 즉시 통계에 반영됨
서브 3	AI와 이어서 대화	추천을 받은 뒤 "아침엔 시간이 안 되는데?" 같은 후속 질문을 이어서 할 수 있음 (최대 5턴)
서브 4	추천 시간 그래프 하이라이트	AI가 고른 시간대를 막대그래프에 ★ 표시로 강조해 말과 그래프를 연결

평가 기준의 '메인 기능 1개 이상, 서브 기능 3개 이상'을 충족한다 (메인 1개 + 서브 4개).

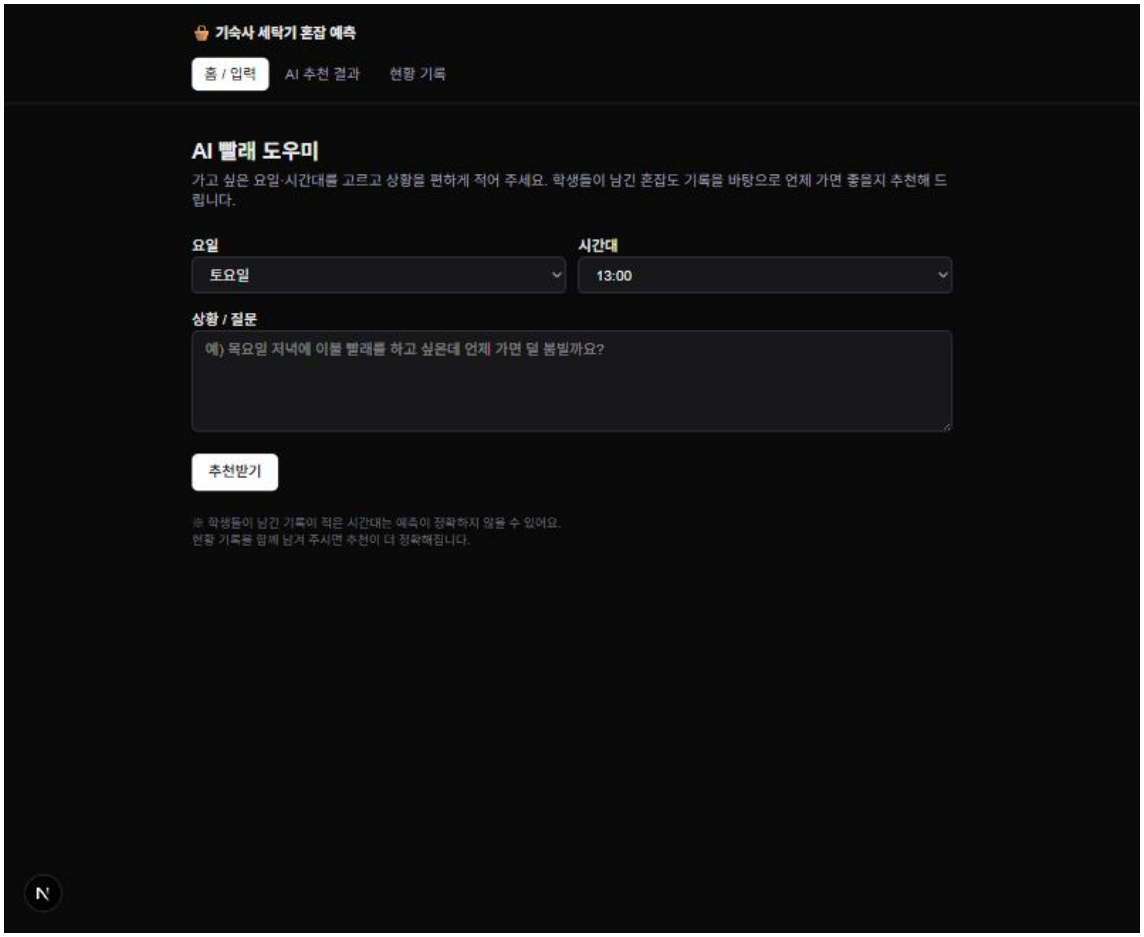
3. 메인 기능 — AI 빨래 도우미

동작 흐름

```
[■■■■■]  ■■ · ■■■ ■■ + ■■■ ■■ ■■
    |
    v  POST /api/recommend
[Next.js ■■]  Supabase■■ ■■ ■■ -> ■■■ ■■■ ■■
    |            + ■■■ ■■  =>  ■■■■ ■■
    v
[Google Gemini]  ■■ ■■ / ■■ ■■ / ■■■ JSON■■ ■■
    |
    v
[■■■■]  ■■■■ ■■ ■■ ■■ + ■■■■ ★ ■■
```

중요: API 키는 브라우저로 나가지 않는다

Gemini 호출은 **서버 라우트 핸들러에서만** 이루어진다. 키는 GEMINI_API_KEY 환경변수로 읽으며, Next.js 규칙상 NEXT_PUBLIC_ 접두사가 없는 변수는 브라우저 번들에 포함되지 않는다. .env.local과 .vercel은 .gitignore에 등록되어 GitHub에도 올라가지 않는다.



[화면 1] 홈 / 입력 — 요일·시간대를 고르고 상황을 자연어로 적는다

실제 응답 예시 (월요일 19시, "월요일 저녁에 빨래하려는데 언제가 좋을까?")

추천: 생각 중이신 19시는 혼잡할 수 있으니, 여유가 되신다면 비교적 한산했던 아침 8시 시간대를 추천드려요.

이유: 19시는 '혼잡' 등급에 평균 여유 세탁기가 1.0대로 자리가 부족할 가능성이 높습니다. 반면 8시는 '여유' 등급에 평균 여유 대수가 4.0대로 비어 있는 편이었어요. 다만 두 시간대 모두 표본이 1건뿐이라 실제 상황은 다를 수 있으니 참고용으로 봐주세요.

표본이 1건뿐이라는 **한계까지 AI가 스스로 밝히도록** 프롬프트에 규칙을 넣었다. 기록이 없는 시간대를 단정하거나, 데이터에 없는 내용을 지어내지 않도록 제약을 둔 것이다.

4. 서브 기능 1 — 시간대별 혼잡도 그래프

선택한 요일의 06시~23시 혼잡도를 막대그래프로 보여 준다. 막대 색은 3단계다.

등급	색	조건 (평균 여유 대수 ÷ 전체 6대)
여유	초록	0.5 이상 — 평균 3대 이상 비어 있음
보통	노랑	0.2 이상 0.5 미만 — 평균 1.2~3대
혼잡	빨강	0.2 미만 — 평균 1.2대 미만
기록없음	회색	해당 요일·시간대에 남겨진 기록이 0건

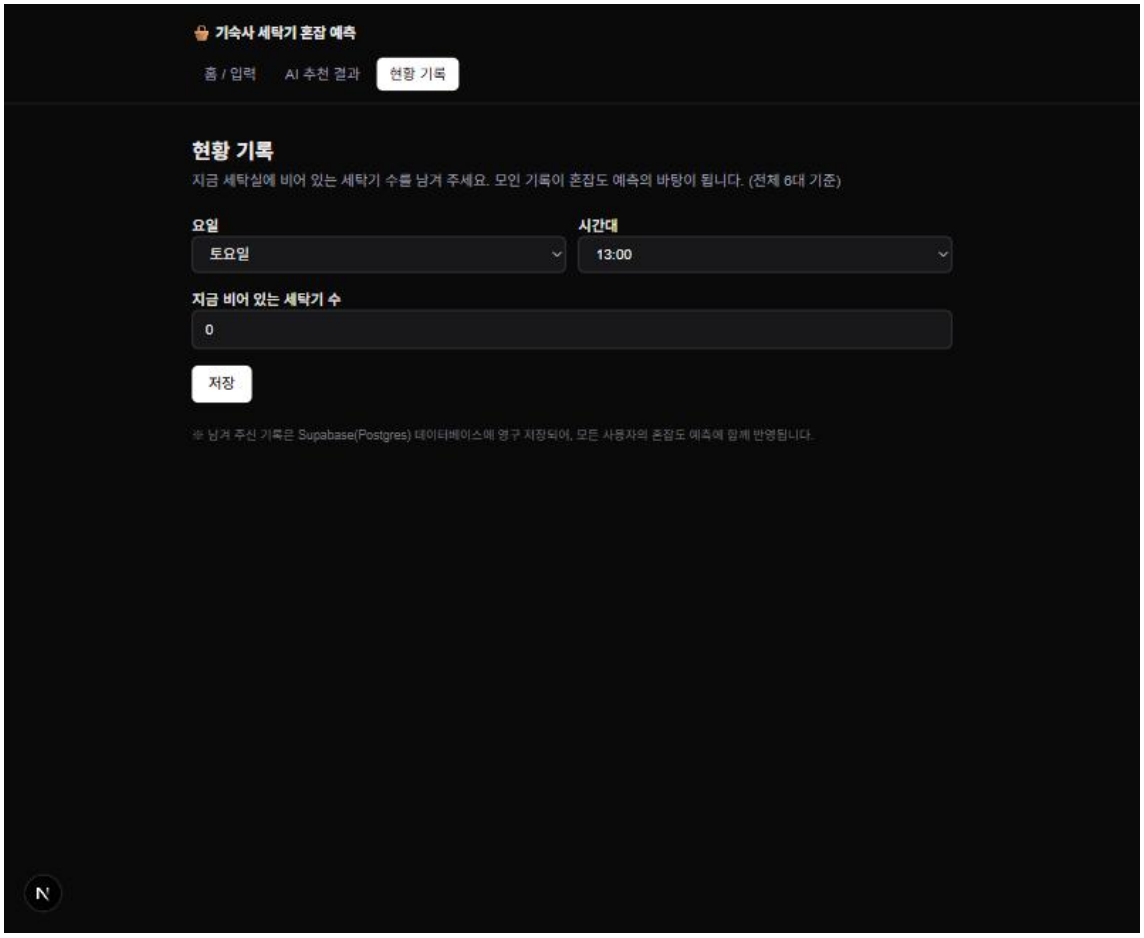
계산 방식

```
1) ■■ (■■, ■■) ■■■ ■■ ■■■
2) '■■ ■■ ■■■ ■■'■ ■■■ ■■
3) ■■ / ■■ 6■  =  ■■ ■■
4) ■■■(0.5 / 0.2)■ ■■■ ■■■ ■■■
```

임계값은 src/lib/config.ts의 CONGESTION_THRESHOLDS 상수 하나로 모아 두어, 기록이 쌓인 뒤 숫자만 바꾸면 기준을 조정할 수 있다.

5. 서브 기능 2 — 현황 기록과 데이터베이스 저장

학생이 세탁실에 갔을 때 비어 있던 세탁기 수를 남기면, 그 기록이 **Supabase(Postgres)**에 저장되어 모든 사용자의 예측에 반영된다. 이 앱에서 데이터가 쌓이는 유일한 통로다.



[화면 2] 현황 기록 — 요일·시간대·비어 있던 대수를 저장한다

6. 데이터베이스 설계 (Supabase)

테이블: laundry_records

컬럼	타입	설명
id	uuid (PK)	자동 생성되는 기본키
day_of_week	smallint	요일 0=일 ~ 6=토 (0~6 범위 제약)
hour	smallint	시각 0~23 (범위 제약)
available_machines	smallint	비어 있던 세탁기 대수 (0 이상 제약)
created_at	timestamptz	기록 생성 시각

(day_of_week, hour) 복합 인덱스를 만들어 요일별 집계 조회를 빠르게 했다.

보안: Row Level Security

테이블에 RLS를 켜고 정책을 **조회(select)와 추가(insert) 두 가지만** 허용했다. 수정·삭제 정책은 만들지 않았으므로, 키가 노출되더라도 남의 기록을 지우거나 바꿀 수 없다.

```
alter table public.laundry_records enable row level security;

create policy "anyone can read records"
  on public.laundry_records for select using (true);

create policy "anyone can insert records"
  on public.laundry_records for insert with check (true);
-- update / delete ■■ ■■ = ■■·■■ ■■
```

스키마와 예시 데이터는 저장소의 supabase/schema.sql, supabase/seed.sql에 들어 있어 누구나 같은 DB를 재현할 수 있다.

7. 서브 기능 3 — AI와 이어서 대화하기

추천을 한 번 받고 끝나는 것이 아니라, 말풍선 아래 입력창에서 **후속 질문**을 이어서 할 수 있다. 예를 들어 "아침에는 시간이 안 되는데 다른 때는 없어?"라고 물으면, AI는 앞선 대화와 혼잡도 데이터를 함께 보고 다른 시간대를 다시 제안한다.

구현에서 신경 쓴 점

• **서버는 대화를 기억하지 않는다.** 배포 환경(서버리스)은 요청마다 다른 인스턴스가 처리할 수 있어 서버 메모리에 대화를 저장하면 유실된다. 그래서 브라우저가 **지금까지의 대화 전체**를 매 요청에 함께 보낸다.

• **혼잡도 표는 대화 내용이 아니라 시스템 지시문에 넣는다.** 대화가 길어져도 데이터가 밀려나지 않고, 토큰도 아낄 수 있다.

• **대화 5턴 제한.** Gemini 무료 등급은 모델당 하루 20회라, 한 대화가 할당량을 다 쓰지 않도록 MAX_CHAT_TURNS로 막았다.

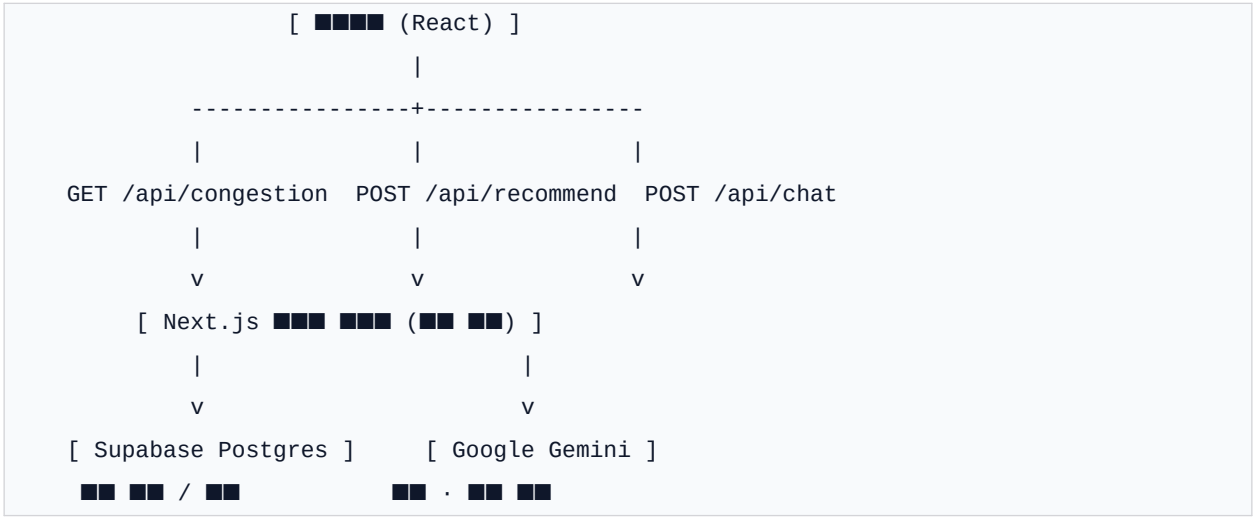
• **요청 검증.** 빈 메시지, 1000자 초과, 마지막 메시지가 사용자 질문이 아닌 경우를 서버에서 걸러낸다.

8. 서브 기능 4 — 추천 시간 그래프 하이라이트

AI가 "8시를 추천한다"고 답하면, 응답에 담긴 recommendedHour 값으로 막대그래프의 해당 시간대에 ★ **표시와 테두리**를 입힌다. AI의 문장과 그래프가 따로 놀지 않고 한눈에 연결된다.

[캡처 준비 중: 02-결과.jpg]

9. 시스템 구조



데이터 접근 계층을 한 곳에 모은 이유

src/lib/data.ts 하나에만 저장소 코드를 두고, 화면과 API는 listRecords() / addRecord() 두 함수만 쓴다. 덕분에 처음에 mock(메모리) 저장소로 만들어 두었다가 Supabase로 바꿀 때 **이 파일 한 개만 고치면** 됐다. 환경변수가 없으면 자동으로 mock으로 돌아가 로컬 개발도 막히지 않는다.

API 라우트

메서드 · 경로	설명
GET /api/records	전체 현황 기록 조회
POST /api/records	새 현황 기록 추가 (Supabase에 저장)
GET /api/congestion?day=	해당 요일의 시간대별 혼잡도 등급
POST /api/recommend	질문 + 혼잡도 → Gemini 추천 (메인 기능)
POST /api/chat	대화 기록 + 혼잡도 → Gemini 후속 답변

10. 시연 따라하기

아래 순서대로 하면 영상 없이도 모든 기능을 그대로 확인할 수 있다. 접속 주소:
https://one-iota-62.vercel.app/

① 메인 기능 — AI 추천 받기

- 홈 화면에서 **요일**을 '월요일', **시간대**를 '19:00'으로 선택한다.
- 질문란에 **"월요일 저녁에 빨래하려는데 언제가 좋을까?"**를 입력한다.
- **[추천받기]** 버튼을 누른다. 결과 화면으로 넘어가며 스피너가 보인다.
- 수 초 뒤 혼잡도 막대그래프와 AI 말풍선이 나타난다. 그래프에서 AI가 추천한 시간대에 ★가 붙어 있는지 확인한다.

② 서브 기능 — AI와 이어서 대화

- 말풍선 아래 입력창에 **"아침에는 시간이 안 되는데 다른 때는 없어?"**라고 입력하고 전송한다.
- AI가 앞선 대화를 기억한 채 다른 시간대를 제안하는지 확인한다.
- 입력창 아래에 **남은 대화 횟수**가 줄어드는 것을 보여 준다.

③ 서브 기능 — 현황 기록과 DB 저장

- 상단 메뉴에서 **[현황 기록]**으로 이동한다.
- 요일 '월요일', 시간대 '19:00', 비어 있는 세탁기 수에 **5**를 입력하고 **[저장]**을 누른다.
- 저장 완료 메시지를 확인한 뒤, 다시 홈에서 월요일 19시로 추천을 받아 본다.
- 방금 넣은 기록이 반영되어 **19시 막대의 색과 등급이 달라진 것**을 보여 준다. 이것이 DB에 실제로 저장되었다는 증거다.

DB 저장을 직접 확인하는 방법: 브라우저에서 https://one-iota-62.vercel.app/api/records 를 열면 저장된 기록이 JSON으로 보인다. 각 기록의 id가 aff08735-b8db-... 같은 **uuid**면 Supabase에서 온 것이다. (메모리 저장소였다면 seed-01 같은 고정 값이 나온다.)

11. 기술 스택 · 배포 · 한계

구분	사용 기술
프레임워크	Next.js (App Router) · TypeScript
스타일	Tailwind CSS
AI	Google Gemini API (@google/genai, gemini-flash 계열)
데이터베이스	Supabase (PostgreSQL) + Row Level Security
배포	Vercel (GitHub 저장소 push 시 자동 배포)

개발하며 해결한 문제

• **API 키 노출 방지** — AI 호출을 전부 서버 라우트로 옮기고, .env.local을 .gitignore에 넣어 저장소에 키가 올라가지 않게 했다.

• **무료 등급 할당량** — Gemini 무료 등급은 모델당 하루 20회다. 재시도 정책에서 429(한도 초과)는 재시도하지 않도록 빼고, 대화 턴 수도 제한해 할당량을 아꼈다.

• **서버리스에서의 상태 유실** — 메모리 저장소는 배포 환경에서 초기화된다는 점을 확인하고 Supabase로 교체했다.

• **오류 메시지 구분** — 잘못된 요청(400)을 인증 오류(401·403)와 함께 처리하던 코드를 분리해, 설정 문제가 "API 키 문제"로 잘못 표시되던 것을 고쳤다.

한계와 개선 방향

• 기록 표본이 아직 적어 일부 시간대는 '기록없음'으로 나온다. 학생들이 꾸준히 입력해야 정확해지는 구조다.

• 같은 사람이 짧은 시간에 여러 번 기록하면 통계가 왜곡될 수 있어, 중복 입력 제한이 필요하다.

• Gemini 무료 등급의 하루 호출 제한 때문에 많은 사용자가 동시에 쓰기는 어렵다.

사이트 · https://one-iota-62.vercel.app/
GitHub · https://github.com/pms09/-
2407 박민석